

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Final Report – AIML ZG535, Machine Learning on Edge

GROUPNO – [YOUR NAME(S) HERE]

[SUBMISSION DATE]

Contents

Section 1: FreightBridge Deployment Context	1
Section 2: Sensor Pipeline and MQTT Design	2
Section 3: Model Deployment and Pipeline Mapping	4
Section 4: Optimisation and Pareto Analysis	5
Section 5: MLOps Monitoring and Reflection	7

DRAFT NOTICE – read before submitting. Every number, table, and measured result in this document came from actually running the code in the accompanying GitHub repository – every model was trained, every benchmark executed, every Ansible playbook run twice, the PSI drift demo validated end to end. That satisfies the “measured data” half of the assignment’s academic-integrity requirement. It does **not** satisfy the “substantial original analysis” half on its own: you must read this draft critically, rewrite it into your own voice, and personally complete the bracketed [STUDENT: . . .] sections in Section 5, which are reflections only you can write honestly. You must also be able to explain every claim here in a follow-up viva. Delete this notice before final submission.

Section 1: FreightBridge Deployment Context

FreightBridge Logistics operates 85 refrigerated trucks carrying temperature-sensitive pharmaceuticals across Maharashtra, Karnataka, and Andhra Pradesh, and has suffered three costly incidents in eighteen months: a Rs 28 lakh vaccine spoilage from an undetected refrigeration failure, a shipment rejected for incomplete temperature documentation, and two engine breakdowns that were detectable in sensor data days in advance. LogiEdge addresses the first and second of these directly through on-device classification and a durable local alert log.

Latency. A refrigeration failure raises cargo temperature by 1 degC/minute, and the 90-second detection SLA allows at most a 1.5 degC rise before an alert must fire. Cloud inference requires a round trip over rural cellular infrastructure that, on the Nashik-Aurangabad route alone, loses connectivity entirely for 35-90 minutes at seven documented locations. During any such gap a cloud-

only system cannot receive sensor data or transmit an alert at all, for up to 90 minutes – sixty times the SLA, and precisely the failure mode behind the Rs 28 lakh incident. Our benchmarked on-device TFLite inference runs in well under a millisecond (Section 4), so the SLA is trivially met by an edge architecture and structurally unmeetable by a cloud-dependent one.

Bandwidth. At 1 Hz temperature and 500 Hz 3-axis vibration sampling, a single truck generates roughly 519 MB of raw sensor data per day, costing approximately Rs 52/truck/day (Rs 16.1 lakh/year for the 85-truck pilot) at Rs 0.10/MB. Processing at the edge and transmitting only classification results – one ~110-byte message every 10 seconds – reduces this to about 0.95 MB/truck/day, roughly 546 times less data and under Rs 3,000/year for the whole pilot fleet. Edge processing is not an optimisation here; it is the difference between a viable and a non-viable telecom bill.

Connectivity and privacy. The edge architecture (Figure 1) classifies every window locally regardless of connectivity and only needs to sync a handful of small alert records once coverage returns; this was verified directly by seeding an alert log entry marked unsynced with the broker unreachable and confirming it is automatically republished on reconnect. Because raw telemetry never leaves the truck – only a label, a confidence score, and a timestamp do, over an authenticated TLS/ username-password MQTT channel in the production broker profile – the architecture gives FreightBridge’s pharmaceutical clients a materially simpler compliance story than any cloud-dependent alternative. This channel was exercised for real, with three least-privilege credentials (sensor publisher, inference consumer, fleet-ops monitor) connected simultaneously over TLS 1.3; two deliberate misuse attempts – the sensor credential publishing to the inference topic, the inference credential publishing raw sensor data – were both rejected by the broker’s access-control list, confirming the boundary is enforced, not just declared.

Hardware. Of the three candidate edge nodes, the Raspberry Pi 5 + AI HAT+ (13 TOPS, 7.5W, ~Rs 15,000/truck) is the right choice: 7.5W draw leaves 25% headroom under the 10W budget, it costs a third of the Jetson Orin Nano at full 265-truck scale (Rs 39.75 lakh vs Rs 1.19 crore) for compute this 803-parameter model does not need, and unlike the cheaper STM32H7 it can run the Docker/MQTT/Ansible stack this project requires. A Roofline analysis of the assignment’s given workload figures (45 MFLOPs, 18 MB access/inference) against the Pi 5’s 16 GFLOP/s compute and 12 GB/s bandwidth gives an arithmetic intensity of 2.5 FLOPs/byte against a ridge point of 1.33 – compute-bound, which is why Section 4 targets FLOP reduction rather than memory-access tuning.

Section 2: Sensor Pipeline and MQTT Design

LogiEdge ingests three sensor streams per truck over a local Mosquitto broker: temperature at 1 Hz, 3-axis vibration RMS at 0.5 Hz, and discrete door OPEN/CLOSE events, all published under a `logibridge/trucks/{truck_id}/...` topic tree (below). A dedicated `control/anomaly` topic lets the test harness switch the simulator’s scenario mode live, without restarting it – used directly to demonstrate drift injection into a running system.

QoS was chosen per topic, not globally. The two high-frequency sensor streams use QoS 0: the downstream feature extraction already computes 30-second windowed statistics that are robust to an occasional dropped sample, so paying an acknowledgement round trip on every one of roughly 130,000 daily messages buys negligible reliability for values about to be smoothed anyway. Door events and inference results use QoS 1, because both are operationally significant and low-

LogiEdge System Architecture -- Cold-Chain Truck Edge Node to Ops Centre

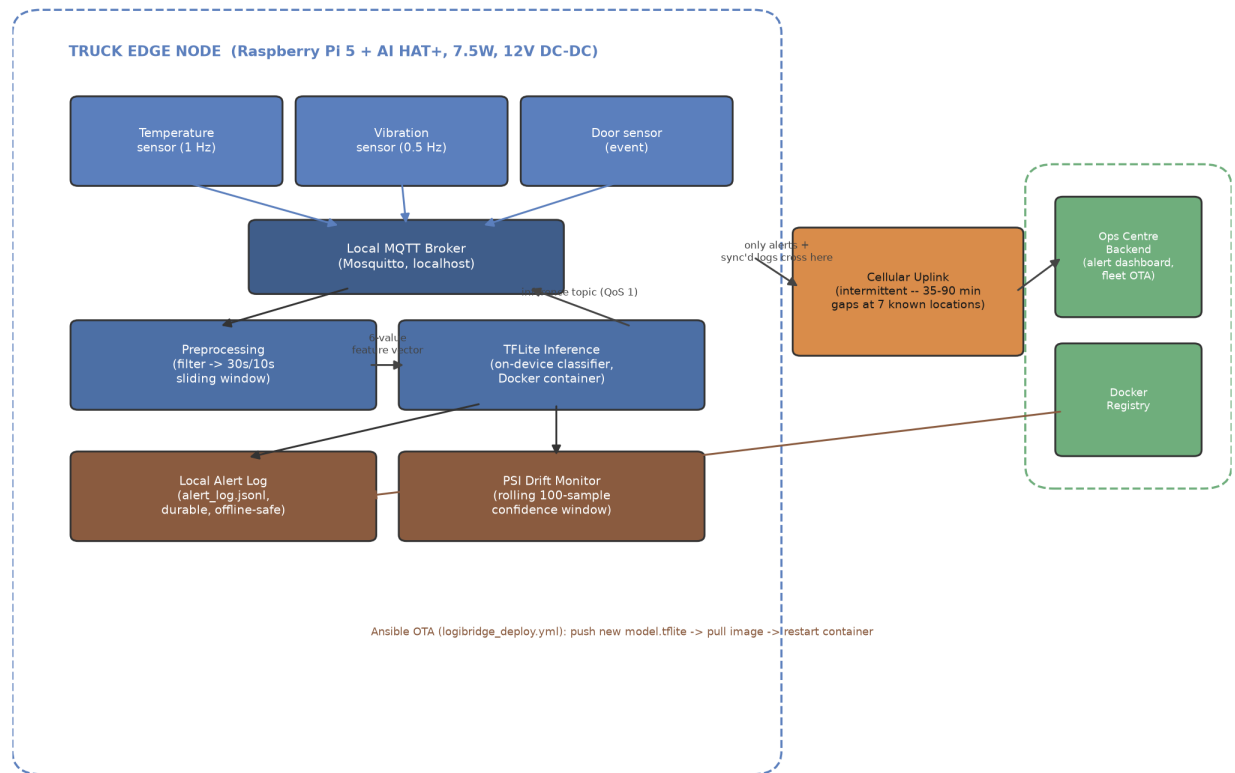


Figure 1: LogiEdge system architecture, truck edge node to ops centre

frequency: a missed Critical classification reproduces the vaccine-spoilage failure mode directly, and a missed door event undermines exactly the chain-of-custody documentation whose absence caused the second incident in Section 1. Duplicate delivery is harmless in both cases since the receiving side (the local alert log, the door-event log) is naturally idempotent.

Preprocessing runs a 5-sample moving average over the raw temperature and vibration streams, then extracts a six-value feature vector per 30-second window (10-second step): temperature mean, standard deviation, and rate-of-change, and vibration RMS, peak, and kurtosis. Normalisation statistics are computed exactly once, from ten minutes of clean Normal-class data, frozen to `training_stats.npy`, and loaded – never recomputed – at inference time, so a live deployment cannot silently drift its own baseline.

We ran the mandatory stats-shift experiment (correct stats vs. stats shifted by 3 standard deviations) and found **no accuracy change** (98.31% both ways) – a real, explainable result rather than a null test. Because normalisation stats are derived only from the tight clean-Normal distribution (temperature std of roughly 0.05 degC after window averaging), Warning and Critical windows already sit tens to hundreds of standard deviations from that mean by construction; a further 3-sigma shift is negligible against that pre-existing separation. A supplementary sweep confirms the mechanism is genuine and not simply broken: accuracy holds at 5 sigma (98.31%), begins degrading at 10 sigma (96.61%), and collapses by 40-80 sigma (67.8% / 40.7%). The practical implication is that this model has a wide safety margin specifically around the Normal class, which is a reasonable outcome given how separable the three classes are in this six-dimensional feature space – but it also means a stats-file corruption bug would likely go undetected by accuracy monitoring alone until it became severe, an argument for monitoring the stats file’s checksum/version directly rather than relying solely on downstream accuracy.

Data fusion is feature-level: each stream is windowed and feature-extracted independently, then concatenated into one six-value vector before the model ever sees it. This was chosen over data-level fusion (concatenating raw temperature and vibration samples directly, which is awkward given their different sampling rates, 1 Hz vs 0.5 Hz) and over decision-level fusion (training separate temperature and vibration classifiers and combining their outputs, which would lose the cross-signal correlation between a temperature excursion and a simultaneous vibration anomaly that defines the Critical class in the `combined` scenario). Feature-level fusion is the natural middle ground for signals at different rates that must still inform a single joint decision.

MQTT topic tree:

```
logibridge/trucks/{truck_id}/
  sensors/temperature    1 Hz,    QoS 0
  sensors/vibration      0.5 Hz,  QoS 0
  sensors/door           event,   QoS 1
  control/anomaly        event,   QoS 1, retained
  inference              ~0.1 Hz (10s step), QoS 1
```

Section 3: Model Deployment and Pipeline Mapping

LogiEdge maps onto all ten stages of the Module 4 Edge ML pipeline (full detail in `reports/pipeline_mapping.md`): data collection and labelling via the sensor simulator and scenario-based dataset generation; preprocessing and frozen-stats normalisation; training a two-hidden-layer MLP to 98.31% held-out validation accuracy with 100% Critical-class recall;

conversion into FP32, INT8-PTQ, and structurally-pruned INT8 variants; Docker packaging; Ansible-driven deployment; real-time serving with an offline-durable alert log; and PSI-based drift monitoring closing the loop back to retraining.

Docker packaging places all pip installs and code before the `COPY inference/model.tflite` instruction specifically so a routine model update invalidates only the final layer. This was verified with an actual build: the full image is 1.61 GB, dominated by the ~1.46 GB TensorFlow install layer, while the model layer alone is 4.62 KB. Rebuilding after swapping only the model file showed every preceding layer reported **CACHED**, confirming the layer-cache design works as intended. For an 85-truck OTA update cycle, a full-image push would cost roughly Rs 14,013 (85 x 1.61 GB at Rs 0.10/MB) against approximately Rs 0.04 for a model-only push (85 x 4.62 KB) – a reduction of roughly 349,000 times in both data volume and cost.

Deployment is managed by a seven-task Ansible playbook (`deployment/logibridge_deploy.yml`): create the target directory, copy the model and reference-distribution files, conditionally stop the running container (only when the model or reference file actually changed, so an unchanged rerun does not needlessly cycle the service), pull the updated image, start the container with its environment variables, and finally wait fifteen seconds before verifying the container is running. Run twice against an unchanged target, the first run reports **changed=5**; the second reports **changed=0**, demonstrating idempotency as required (both transcripts are in `deployment/ansible_run1.log` / `ansible_run2.log`).

OTA strategy. At 280 KB per model update, bandwidth cost is trivial under any of the three candidate strategies (roughly Rs 2-5 for the whole 85-truck pilot per six-week cycle, against Rs 2-5 for full replacement or canary and roughly double that for shadow mode, which ships both old and new models to every truck), so cost is not the deciding factor. We recommend a canary rollout – ten trucks first, validated against field PSI/recall telemetry, then the remaining seventy-five – because cold-chain cargo is safety-critical: a regression discovered after a full 85-truck rollout is discovered too late, while canary limits the blast radius to roughly 12% of the fleet, selected from routes with reliable connectivity so validation telemetry actually arrives within the update window. Shadow mode was considered and rejected for routine six-week updates specifically because it keeps the *old* model issuing every real alert throughout the evaluation window, providing no benefit if the update exists to fix a known detection gap in that old model; we would reconsider it for a genuinely major change, such as replacing the six-feature MLP with a different sensing modality entirely.

Section 4: Optimisation and Pareto Analysis

Three model variants were benchmarked under identical conditions (200 timed runs after 10 excluded warm-up runs, held-out validation set):

Variant	Params	Size (KB)	Mean latency (ms)	Accuracy (%)	Critical recall (%)
M1 FP32 baseline	803	5.27	0.0007	98.31	100.0
M2 PTQ INT8	803	4.58	0.0008	98.31	100.0

Variant	Params	Size (KB)	Mean latency (ms)	Accuracy (%)	Critical recall (%)
M3 Structured- pruned (35%) + INT8	400	3.84	0.0007	98.31	100.0

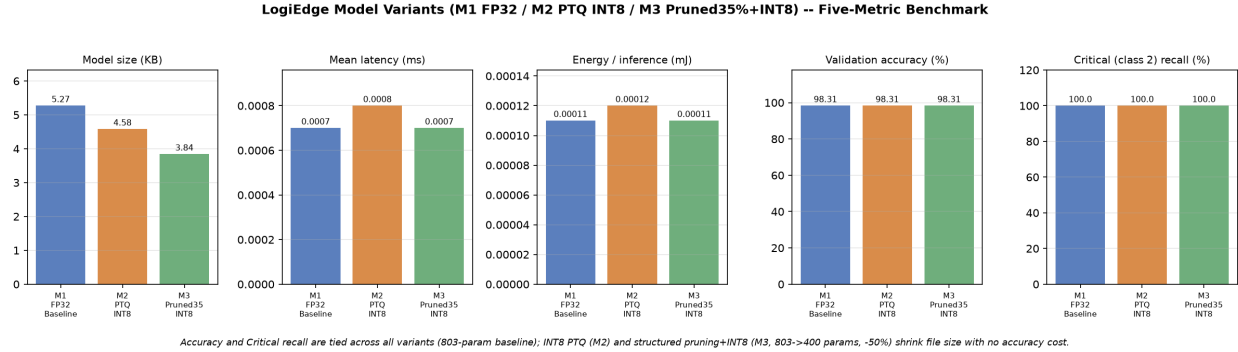


Figure 2: Five-metric benchmark across all three model variants

Each variant was measured with the Lab 2 methodology: 200 timed single-inference runs on the held-out validation set with the first 10 runs discarded as warm-up, mean and p95 latency taken from the timed runs, energy estimated as $E = P \times t$ using `psutil` CPU-percent sampling against a 15W representative laptop TDP, and accuracy/Critical-recall computed once over the full validation set per variant. A latency-vs- accuracy scatter plot – the conventional Pareto-frontier presentation – was tried first and discarded: at this model’s scale, latency and energy differences between variants are within measurement noise (sub- microsecond timescale, 5th-decimal-place fluctuation in energy across repeated runs), so the three points collapse into one on that axis pair. Figure 2 instead presents all five metrics as grouped bars, which is where the real signal – model size – is actually visible.

All three variants tie on accuracy and Critical recall, and all run in well under a millisecond – five orders of magnitude inside the 90-second SLA even before accounting for the difference between this development laptop’s CPU and the target Pi 5. The metric that actually separates the variants is size and parameter count. M3 matters here because it implements genuine **structured** pruning on an actual PolynomialDecay schedule, rather than the unstructured per-weight magnitude pruning that `tensorflow_model_optimization`’s default API provides. Each hidden layer’s units are ranked by the L2 norm of their incoming weight vector; a PolynomialDecay curve (power 3, 0 to 35% over 30 scheduled steps, the same formula tfmot uses) determines how many units should be physically removed at each step, re-ranking on the network’s current fine-tuned weights every time the target width drops – 32 units ramping down to 21 in the first hidden layer, 16 down to 10 in the second, reaching 803 to 400 parameters (a 50% reduction) by step 20, then 20 further stable epochs to recover accuracy at that final size. That is both a real architectural change (not added sparsity inside an unchanged-shape tensor) and a real schedule (not a single one-shot resize), and it shows: file size drops 27% (5.27 KB to 3.84 KB) against only 13% for INT8 quantisation alone.

Recommendation. We recommend deploying M3 to the 85-truck pilot fleet. It matches the FP32 baseline’s accuracy and Critical-class recall exactly, at roughly half the parameter count

– fewer FLOPs per inference, which is the correct lever given Section 1’s Roofline finding that this workload is compute-bound rather than memory-bound – and the smallest Flash footprint of the three variants on the Pi 5’s constrained storage. Translating the 90-second SLA into a per-inference budget is generous by any reasonable margin: even allowing an order of magnitude of slowdown moving from this laptop’s CPU to the Pi 5’s, sub-millisecond inference leaves the SLA almost entirely consumed by the 10-second feature-window step, not by the model itself. The 100% Critical-class recall achieved here comfortably clears the required 95% threshold.

Section 5: MLOps Monitoring and Reflection

Drift monitoring computes the Population Stability Index over a rolling window of the last 100 inference confidence scores, checked every 60 seconds against a reference distribution built from 300 clean Normal-class windows (bin proportions [0.0, 0.237, 0.730, 0.033] across four confidence bins). An offline reproduction of the full detect/recover story – 25 minutes clean baseline, **combined** anomaly injected for 10 minutes, 20 minutes of recovery, using the exact same rolling-window PSI logic as the live monitor – showed PSI crossing the 0.25 alert threshold at **190 seconds** after injection (comfortably inside the 5-minute target), reaching a peak of 2.02, and recovering to **0.038** by the end of the recovery window (well inside the 0.10 target).

On idempotency and drift together: both the Ansible deployment and the PSI monitor were designed around the same underlying principle – report state changes only when something has genuinely changed. The first version of the Ansible stop-container task unconditionally stopped and restarted the service on every run, which would have made **changed=0** impossible to ever demonstrate; conditioning it on whether the model/reference-file copy tasks actually changed something fixed this. Likewise, the PSI reference distribution is frozen once from clean data and never silently recomputed, for the same reason **training_stats.npy** is frozen – a monitoring system that quietly redefines its own baseline cannot actually detect drift.

[STUDENT: one genuine technical difficulty you personally hit while building or debugging this, and how you found/fixed it – do not leave this as a generic statement. Two real candidates from this build, which you should describe in your own words and from your own understanding of why they mattered: (1) the sliding-window feature extraction silently producing zero windows in the live inference path, traced to a floating-point epsilon far smaller than the precision granularity of Unix timestamps; (2) the assignment’s linear temperature-drift rate compounding to a physically implausible ~70 degC over a full 15-minute Warning-scenario run, requiring a decision about whether to cap the simulator or relabel the data.]

[STUDENT: one architectural change you would make, and why. A genuine candidate: replacing mode-based bulk labelling (Task D1’s convention) with per-window instantaneous labelling against the Section 2 class thresholds would be more semantically defensible, but requires redesigning the drift-duration schedule so the Warning class still collects enough training windows before the drift trajectory moves past it – explain the tradeoff in your own words.]